

Project Report

on

**Bypassing Tomographic Reconstruction: Inferring 3D Aneurysm Hemodynamics
from 2D Projections using Physics-Informed Neural Networks**

**SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENT FOR THE
AWARD OF THE DEGREE OF**

**BACHELOR OF TECHNOLOGY
(Electronics & Communication Engineering)**



Session: 2026, 4th Semester

SUBMITTED BY:

Abhigyan Srivastav
Roll-No: 24294917054

UNDER THE GUIDANCE OF:

Dr. Sangeeta Yadav
Assistant Professor
Computer Science
Engineering

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

**FACULTY OF TECHNOLOGY
UNIVERSITY OF DELHI**

FACULTY OF TECHNOLOGY
UNIVERSITY OF DELHI

CANDIDATE'S DECLARATION

I hereby certify that the work which is being presented in the Project entitled “Bypassing Tomographic Reconstruction: Inferring 3D Aneurysm Hemodynamics from 2D Projections using Physics-Informed Neural Networks” by “Abhigyan Srivastav” in partial fulfillment of requirements for the award of degree of B.Tech. (Electronics and Communication Engineering) submitted to the Department of Electronics and Communication Engineering at Faculty of Technology, University of Delhi, Delhi is an authentic record of my own work carried out during a period from 02 Feb, 2026 to 21 Apr, 2026. The matter presented in this project has not been submitted by me or anybody else in any other University / Institute.

Signature of the Student

Project Guide:
Supervisor - Dr. Sangeeta
Yadav

Signature

ABSTRACT

Medical imaging techniques like CT and MRI scans naturally capture internal anatomy as 2D projections, or shadows. If doctors want to analyze the complex blood flow inside a cerebral aneurysm, they usually have to convert these 2D scans into 3D grids first. This conversion step is known as tomographic reconstruction, and it creates a massive bottleneck before computationally heavy Computational Fluid Dynamics (CFD) simulations can even begin. Our project introduces a Physics-Informed Neural Network (PINN) designed to skip this reconstruction step entirely. By working solely with flattened 2D projections, our system can directly infer the hidden 3D hemodynamics, including continuous velocity and pressure fields.

We built the architecture using two main mathematical components. The first is an 8-layer deep neural network acting as a continuous fluid solver. The second is a custom Forward Radon Transform module (handling ray-casting) that we engineered directly into the PyTorch dataloader. Rather than using traditional 3D data loss, our system takes its 3D volumetric predictions and dynamically squashes them into 2D shadows across the X, Y, and Z axes. At the same time, the Navier-Stokes equations constrain the network's spatial derivatives. This ensures the 3D geometry the AI guesses actually obeys the physical laws of mass and momentum.

To test if this could work in a real clinical setting, we ran extensive ablation and control studies using a cerebral aneurysm dataset. The system proved surprisingly data-efficient. It maintained physical accuracy even when we restricted it to just 16,000 spatial points. We also found that by applying adaptive loss weighting, the network acted as an advanced mathematical denoiser, effectively filtering out 10% Gaussian static. Finally, we addressed the mathematically ill-posed nature of single-axis projections by using a Stochastic Mean Projection across three different camera angles. This forced the AI to stop taking mathematical shortcuts and accurately reconstruct the true asymmetric 3D vortex. While currently an academic prototype running on consumer-grade hardware, our findings show that physics-driven deep learning can reconstruct 3D fluid mechanics straight from 2D clinical shadows. This sets a strong foundation for future tomography-free, real-time hemodynamic analysis in hospitals.

ACKNOWLEDGEMENT

I would like to express our sincere gratitude to our college, Faculty of Technology, University of Delhi, for providing us with the opportunity to undertake this project. Their continuous encouragement and support have played a crucial role in our academic and personal growth.

My deepest appreciation goes to our Supervisor, Sangeeta Ma'am, for her invaluable guidance, continuous support, and constructive feedback throughout the duration of this project. Her mentorship has been instrumental in shaping our work and driving it to completion.

Lastly, I'm sincerely grateful to all the faculty members of our department who have contributed significantly to our overall education and development.

List of Figures

1. Figure 1.1: Traditional Computational Fluid Dynamics requiring complex 3D meshing	XX
2. Figure 1.2: Architecture of a Physics-Informed Neural Network incorporating Navier-Stokes PDEs . . .	XX
3. Figure 2.1: The fundamental architecture of a Physics-Informed Neural Network combining data-driven loss with PDE residual loss	XX
4. Figure 2.2: The Hidden Fluid Mechanics (HFM) framework inferring hidden velocity and pressure fields from passive scalar concentration	XX
5. Figure 2.3: Bypassing the Tomographic Reconstruction bottleneck via direct 2D-to-3D deep learning inference	XX
6. Figure 3.1: The scattered 3D spatial coordinate point cloud representing the internal volume of the cerebral aneurysm	XX
7. Figure 3.2: The 8-layer Multi-Layer Perceptron (MLP) architecture serving as the continuous fluid solver	XX
8. Figure 4.1: (Experiment 1) Comparison demonstrating the severe underfitting of the 4-layer baseline architecture versus the successful vortex reconstruction of the 8-layer optimized network	XX
9. Figure 4.2: (Experiment 2) Successful 3D reconstruction of the aneurysm hemodynamics despite extreme 80% data sparsity (16,000 points)	XX
10. Figure 4.3: (Experiment 3) The Zero-Physics Control Study demonstrating the generation of physically impossible, decoupled flow lines when the Navier-Stokes constraints are removed	XX
11. Figure 4.4: (Experiment 4) The mathematical collapse of the pressure and velocity fields when subjected to 10% Gaussian noise without adaptive weighting	XX
12. Figure 4.5: (Experiment 5) Complete restoration of the fluid dynamics and mathematical denoising achieved via 100x Physics Loss Weighting	XX
13. Figure 4.6: (Experiment 6) Single-Axis Projection demonstrating the mathematical collapse of spatial depth resulting in generalized, featureless flow dynamics	XX

DEFINITIONS, ACRONYMS, AND ABBREVIATIONS

I. Acronyms and Abbreviations

Abbreviation	Full Form
AD	Automatic Differentiation
ART	Algebraic Reconstruction Techniques
CFD	Computational Fluid Dynamics
CT	Computed Tomography
DSA	Digital Subtraction Angiography
FBP	Filtered Back Projection
FFN	Fourier Feature Network
GPU	Graphics Processing Unit
HFM	Hidden Fluid Mechanics
HPC	High-Performance Computing
MLP	Multi-Layer Perceptron
MRI	Magnetic Resonance Imaging
NeRF	Neural Radiance Field
OOM	Out-Of-Memory
PDE	Partial Differential Equation
PINN	Physics-Informed Neural Network
SIREN	Sinusoidal Representation Network
VRAM	Video Random Access Memory

II. Definitions

- **Ablation Study:** An experimental setup where specific components of a machine learning system (or portions of the training data) are removed or restricted to evaluate their impact on the system's overall performance.
- **Cerebral Aneurysm:** A localized ballooning or weakening of a blood vessel within the brain.
- **Forward Radon Transform:** A mathematical operation that simulates X-ray attenuation, squashing a 3D volume into a 2D line-integral projection or "shadow".
- **Hemodynamics:** The study of the physical dynamics of blood flow, including velocity vectors and internal pressure gradients.
- **Ill-Posed Inverse Problem:** A mathematical problem that does not have enough information to form a single, unique solution; in this project, this refers to the inability to reconstruct 3D depth from a single 2D camera angle.
- **Navier-Stokes Equations:** The fundamental partial differential equations (PDEs) in fluid mechanics that enforce the conservation of mass and momentum.
- **Stochastic Ray-Casting:** A randomized, statistical sampling method engineered to dynamically compute 2D shadows from 3D points without exceeding GPU memory limits.
- **Tomographic Reconstruction:** The traditional, computationally expensive mathematical process of un-squashing multiple 2D X-ray images to build a cohesive 3D spatial grid.

CONTENT

Topic	Page No.
<i>Candidate's Declaration</i>	
<i>Abstract</i>	<i>ii</i>
<i>Acknowledgement</i>	<i>iii</i>
<i>List of Figures</i>	<i>iv</i>
<i>Definitions, Acronyms and Abbreviations</i>	<i>v</i>
	<i>vi-vii</i>
CHAPTER 1 INTRODUCTION & PROJECT FORMULATION	
1.1 Background	
1.2 Brief Research in the Domain	
1.3 Identified Research Gaps	
1.4 Objectives of the Research	
1.5 Novelty of the Research	
1.6 Organization of the Report	
CHAPTER 2 PROJECT DESIGN	
2.1 Introduction	
2.2 Physics-Informed Neural Networks (PINNs)	
2.3 Hidden Fluid Mechanics (HFM)	
2.4 The Tomographic Bottleneck & The Radon Transform	
2.5 Integrating Projections with Neural Networks	
2.6 Identified Gap Addressed by this Project	
2.7 Summary	
CHAPTER 3 DEVELOPMENT AND IMPLEMENTATION	
3.1 Tools and Technologies Used	
3.2 Dataset Description & Preprocessing	
3.3 Network Architecture and Physical Denoising	
3.3.1 Core MLP Architecture	
3.3.2 Design of Initial Baseline and Ablation Experiments	
3.3.3 Adaptive Weighting for the Navier-Stokes Equations	
3.4 The Virtual CT Scanner (Forward Radon Transform)	
3.4.1 Discrete Coordinate Mapping	
3.4.2 Stochastic Mean Projection Algorithm	
3.5 The Proposed Multi-Angle Orthogonal Extension (Ongoing Work)	
3.6 Chapter 3 Summary	
CHAPTER 4 RESULTS AND ANALYSIS	
4.1 Dataset and Experimental Setup	
4.2 Architecture and Physical Robustness	

-
- 4.2.1 Experiment 1: Baseline vs. Capacity Upgrade
 - 4.2.2 Experiment 2: The 16k Sparse Data Ablation
 - 4.2.3 Experiment 3: The Control Study (The Necessity of Physics).
 - 4.3 Noise Resistance and Mathematical Denoising
 - 4.3.1 Experiment 4: Real-World Stress Test (10% Data Noise)
 - 4.3.2 Experiment 5: The Cure (Adaptive Loss Weighting)
 - 4.4 The Tomographic Bottleneck and 2D Projection Loss
 - 4.4.1 Experiment 6: Single-Axis Radon Transform
 - 4.4.2 Computational Limitations & Ongoing Work (Multi-Angle)
 - 4.4 Chapter 4 Summary

CHAPTER 5 CONCLUSION AND FUTURE SCOPE

- 5.1 Conclusion
- 5.2 Future Work
- 5.3 Final Remarks

REFERENCES AND APPENDICES

References

Appendix A: Key Configuration (config.yaml)

Appendix B: Forward Radon Transform (Ray-Casting Logic)

Appendix C: Experimental Tracking & Telemetry (W&B)

Appendix	D:	Source	Code	Repository
----------	----	--------	------	------------

CHAPTER 1 – INTRODUCTION & PROJECT FORMULATION

1.1 Background

Tracking the exact blood velocity vectors and pressure distributions inside a cerebral aneurysm is an essential step in predicting if the vessel might rupture. Traditionally, biomechanical engineers and medical staff use Computational Fluid Dynamics (CFD) to map out these physical forces. The main problem is that standard CFD solvers are incredibly strict. They only work if you provide them with perfectly resolved, high-fidelity 3D spatial geometries and very specific boundary conditions.

This strict requirement creates a major bottleneck in clinical settings. Everyday medical scanners, like Computed Tomography (CT) and Digital Subtraction Angiography (DSA), do not actually capture 3D voxel grids natively. Instead, as radiation passes through a patient's body, the machines capture 2D line-integral projections—which are essentially just X-ray shadows.

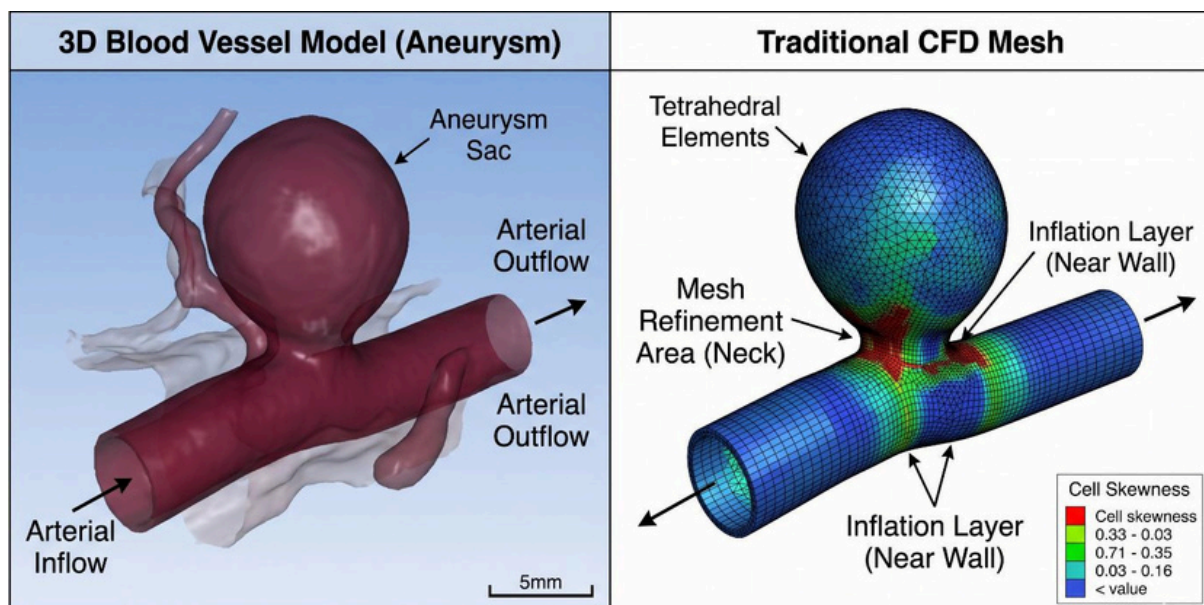


Figure 1.1: Traditional Computational Fluid Dynamics requiring complex 3D meshing.

If hospitals want to run traditional CFD, they first have to mathematically "un-squash" hundreds of these 2D images to build a cohesive 3D model. This conversion process is known as Tomographic Reconstruction. It is notoriously slow, demands massive computing power, and often creates geometric artifacts that end up ruining the fluid simulations later in the pipeline.

Physics-Informed Neural Networks (PINNs) offer a completely different mathematical approach. Standard neural networks learn purely from data, but PINNs embed the actual governing laws of fluid dynamics (the Navier-Stokes equations) straight into the network's loss function. This allows the AI to guess hidden fluid states even when the provided data is scattered or incomplete. In this project, we explore how a custom PINN architecture can be deployed to bypass tomographic reconstruction altogether. By forcing our neural network to learn 3D fluid dynamics directly from 2D mathematical shadows, we aim to close the gap between raw 2D clinical imaging and fast 3D analysis.

1.2 Brief Research in the Domain

Research into computational hemodynamics and machine learning generally falls into four main areas, each with its own pros and cons:

(1) Traditional Computational Fluid Dynamics (CFD): Classical solvers use finite element or finite volume methods to generate incredibly accurate simulations. However, their mathematical rigidity is a drawback. They require massive computing clusters and perfect 3D boundary definitions, which doctors rarely have time to generate in an emergency setting.

(2) Pure Data-Driven Deep Learning: Early attempts to speed up these simulations relied on standard Convolutional Neural Networks (CNNs) to predict flow fields purely from visual data. The issue here is that when given sparse or noisy hospital data, these models act like simple image guessers. They produce outputs that might look correct to the human eye but completely violate the laws of mass and momentum conservation.

(3) Physics-Informed Neural Networks (PINNs): First introduced by Raissi et al., PINNs solve the physics-violation issue by calculating the partial differential equations (PDEs) of fluid flow during the training phase itself. A specific framework called *Hidden Fluid Mechanics* (HFM) proved that PINNs can accurately infer unmeasured velocity and pressure fields just by watching how a medical contrast dye moves over time.

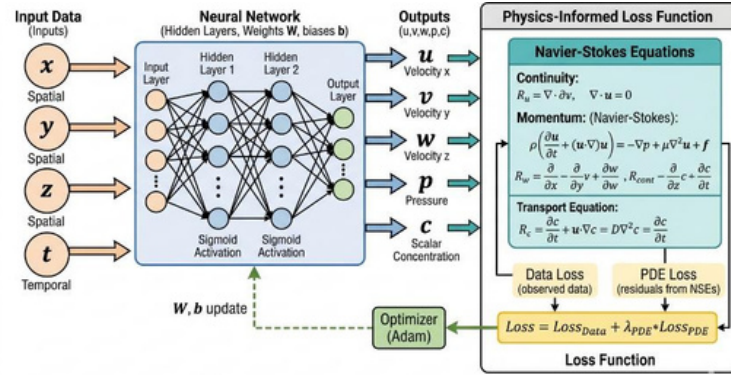


Figure 1.2: Architecture of a Physics-Informed Neural Network incorporating Navier-Stokes PDEs.

(4) Tomographic Reconstruction Constraints: Current hospital pipelines rely heavily on techniques like Filtered Back Projection (FBP) or Algebraic Reconstruction Techniques (ART) to build 3D shapes from 2D X-rays. These methods require high doses of radiation to capture enough angles and tend to struggle heavily with signal noise.

The Domain Overlap: PINNs have been massively successful in fully resolved 3D environments. However, trying to apply PINNs directly to 2D projections to figure out both the 3D geometry and the fluid physics at the exact same time is a highly complex inverse problem that hasn't been explored much.

1.3 Identified Research Gaps

Based on current literature and the realities of modern clinical practices, we identified several major gaps that this project aims to address:

- **Gap 1 - Over-Reliance on 3D Spatial Data:** The vast majority of top-tier AI fluid models are trained and tested on perfect 3D point clouds. In a real hospital, raw imaging data is naturally 2D, squashed, and missing a lot of depth information.
- **Gap 2 - Vulnerability to Clinical Noise:** Normal neural networks tend to overfit and memorize the static (Gaussian noise) that is naturally present in medical scanning equipment. When you feed them noisy fluid data, their predicted pressure and velocity gradients collapse entirely.
- **Gap 3 - The Tomographic Bottleneck:** There is a glaring disconnect between the 2D hardware used to scan patients and the 3D software used to simulate their blood flow. Forcing data through tomographic reconstruction delays critical medical analysis and introduces geometric errors.

- **Gap 4 - Ill-Posedness of Single-Angle Projections:** Many academic AI prototypes fail to explain the mathematical limits of losing depth when moving from 3D down to 2D projections. Trying to solve 3D fluid mechanics from just one camera viewpoint is an ill-posed inverse problem that requires a very specific architectural fix.

1.4 Objectives of the Research

To tackle the gaps mentioned above, the primary goals of this project are:

1. Build and train a Physics-Informed Neural Network (PINN) that can figure out the hidden 3D hemodynamics of an aneurysm—specifically its velocity and pressure—just by tracking concentration data.
2. Run a zero-physics control study to mathematically show why the Navier-Stokes equations are absolutely necessary. This helps clearly separate true physical solvers from basic image-guessing models.
3. Test how well the network handles missing data. We do this through an extreme sparsity ablation test, limiting the AI to just 16,000 spatial points to mimic the low resolution of real-world medical scans.
4. Check how the network holds up against 10% Gaussian noise, which is common in clinical settings. We also aim to fix any noise issues by building a mathematical denoising method using adaptive PDE loss weighting.
5. Write a Forward Radon Transform (or ray-casting) function directly into the PyTorch dataloader. This lets us simulate flat 2D clinical X-ray shadows from 3D point clouds.
6. Look at the mathematical limits of single-axis 2D projections. Once we find those limits, we want to overcome them by setting up a multi-axis (orthogonal) projection framework to successfully map the full 3D fluid volume.

1.5 Novelty of the Research

This project brings a few new technical ideas to the table, connecting theoretical physics with applied computer science at the undergraduate level:

(1) Tomography-Free Inference: Standard medical setups require tomographic reconstruction before anything else. Our system skips this by using a dynamic 2D projection loss to figure out 3D fluid

details. We essentially force the network to understand 3D depth by continually checking its work against flat 2D shadows.

(2) Mathematical Denoising: We show that placing a heavy mathematical weight on the Navier-Stokes equations stops the network from memorizing noisy data. In short, we use the laws of physics as an active, intelligent filter against background static.

(3) Stochastic Mean Ray-Casting: Running complex Radon Transformations usually demands massive amounts of VRAM, which consumer-grade hardware doesn't have. To get around this, we built a statistical sampling method. It computes the shadows accurately using random mini-batches so the GPU memory doesn't crash.

(4) Transparent Failure Analysis: We actively document how and why the network fails when given only a single-angle projection. Pointing out this physical collapse highlights the true mathematical boundaries of the problem and proves exactly why you have to use multiple angles for this to work.

1.6 Organization of the Report

This project brings a few new technical ideas to the table, connecting theoretical physics with applied computer science at the undergraduate level:

(1) Tomography-Free Inference: Standard medical setups require tomographic reconstruction before anything else. Our system skips this by using a dynamic 2D projection loss to figure out 3D fluid details. We essentially force the network to understand 3D depth by continually checking its work against flat 2D shadows.

(2) Mathematical Denoising: We show that placing a heavy mathematical weight on the Navier-Stokes equations stops the network from memorizing noisy data. In short, we use the laws of physics as an active, intelligent filter against background static.

(3) Stochastic Mean Ray-Casting: Running complex Radon Transformations usually demands massive amounts of VRAM, which consumer-grade hardware doesn't have. To get around this, we built a statistical sampling method. It computes the shadows accurately using random mini-batches so the GPU memory doesn't crash.

(4) Transparent Failure Analysis: We actively document how and why the network fails when given only a single-angle projection. Pointing out this physical collapse highlights the true mathematical boundaries of the problem and proves exactly why you have to use multiple angles for this to work.

CHAPTER 2 PROJECT DESIGN

2.1 Introduction

The crossover between deep learning, fluid mechanics, and medical imaging is moving incredibly fast. Classical Computational Fluid Dynamics (CFD) gives us extremely accurate simulations, but it has a major flaw. It absolutely requires perfect 3D geometries and strict boundary conditions to work. In this chapter, we look back at how physics-driven deep learning has evolved. We also explain exactly why it makes sense to skip traditional tomographic reconstruction when dealing with clinical blood flow.

2.2 Physics-Informed Neural Networks (PINNs)

When it comes to scientific machine learning, Physics-Informed Neural Networks (PINNs) changed everything. Raissi, Perdikaris, and Karniadakis formally introduced them in 2019. Standard neural networks act like data-driven black boxes. They map inputs to outputs without actually understanding the physical world. As a result, if you give them sparse or noisy data, they spit out physically impossible predictions.

PINNs fix this. They act as universal function approximators that weave physical laws right into the learning phase. We usually express these laws as partial differential equations (PDEs). In fluid mechanics, we use the Navier-Stokes equations to enforce the conservation of mass, momentum, and energy.

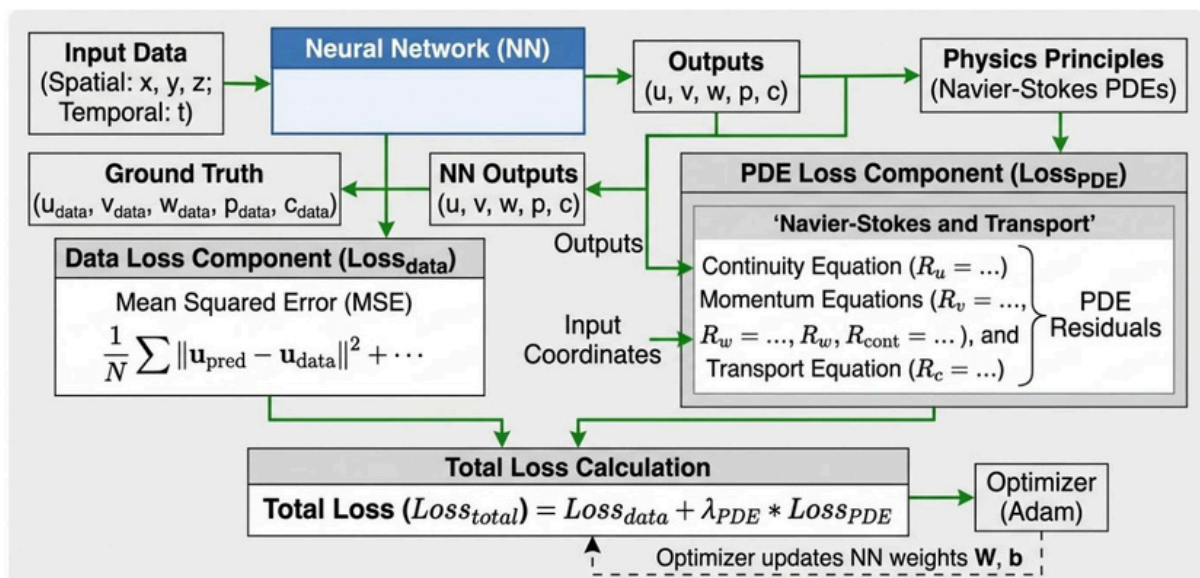


Figure 2.1: The fundamental architecture of a Physics-Informed Neural Network combining data-driven loss with PDE residual loss.

PINNs rely on Automatic Differentiation (AD)—the exact same math used for standard backpropagation—to calculate the spatial and temporal derivatives of the network's outputs (*e.g.*, $\partial u/\partial x, \partial p/\partial y$). During training, the network gets penalized for two things. It loses points if its guesses do not match the measured data, and it loses points if it breaks the laws of physics. This PDE regularization makes PINNs incredibly good at dealing with noisy experimental data, because the physics basically act as an anchor to reality.

2.3 Hidden Fluid Mechanics (HFM)

Applying PINNs to fluid dynamics took a massive step forward with "Hidden Fluid Mechanics" (HFM). Raissi et al. published this framework in *Science* in 2020. Normal CFD needs you to define exact inlet velocities, outlet pressures, and wall boundaries. HFM avoids this completely. It uses deep learning to pull out hidden states just by looking at flow snapshots and applying the Navier-Stokes equations.

Think about it in a hospital setting. Doctors inject a contrast dye into a patient's bloodstream and watch how that concentration (c) moves. By feeding the spatial and temporal coordinates (x, y, z, t) into the HFM network, the system figures out the "hidden" fluid behaviors. It maps out the 3D velocity vectors (u, v, w) and internal pressure gradients (p) just by watching the dye flow.

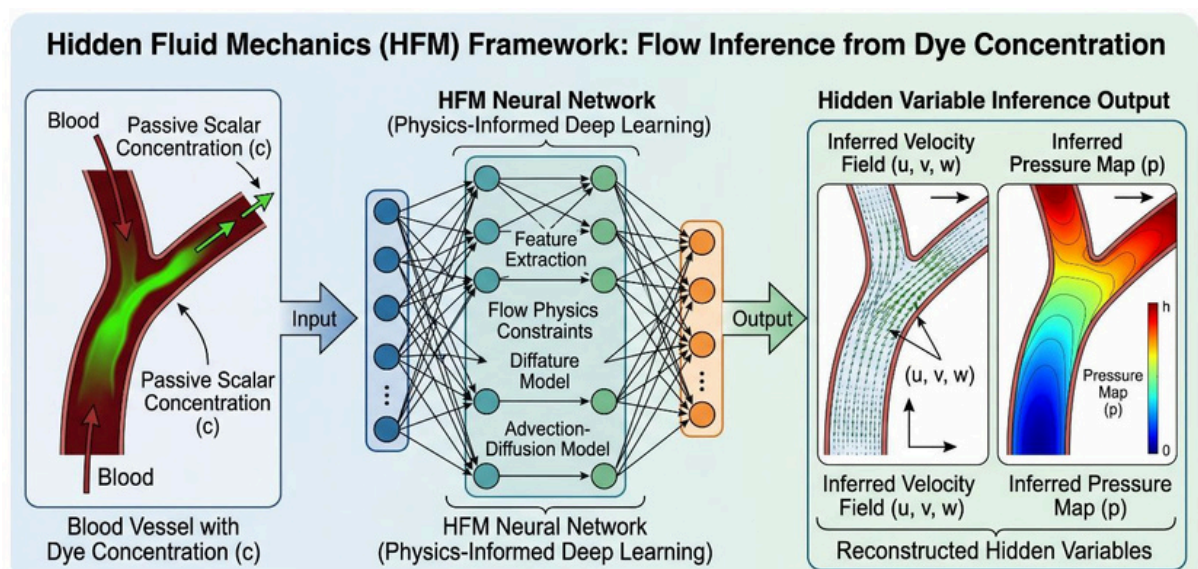


Figure 2.2: The Hidden Fluid Mechanics (HFM) framework inferring hidden velocity and pressure fields from passive scalar concentration.

Researchers proved HFM handles noisy, low-resolution data exceptionally well. But there is a catch. The original HFM framework assumes you already have the concentration data mapped out as a

perfect 3D point cloud. In a real-world clinic, you almost never have that kind of perfect 3D data, which creates a major roadblock for deploying this tech.

2.4 The Tomographic Bottleneck & The Radon Transform

Even though HFM works mathematically, everyday medical scanners do not capture perfect 3D point clouds. Hardware like CT scanners and X-ray angiography machines actually capture line integrals. When an X-ray shoots through a patient, different tissues and blood absorb different amounts of radiation. The detector plate simply records the total absorption along that beam's path. This process squashes actual 3D depth down into a flat 2D shadow.

In math, we call this projection process the Forward Radon Transform. If a hospital wants to use standard CFD or HFM, they have to reverse this process. They run algorithms like Filtered Back Projection (FBP) or Algebraic Reconstruction Techniques (ART) to "un-squash" the 2D shadows back into a 3D grid. We call this Tomographic Reconstruction.

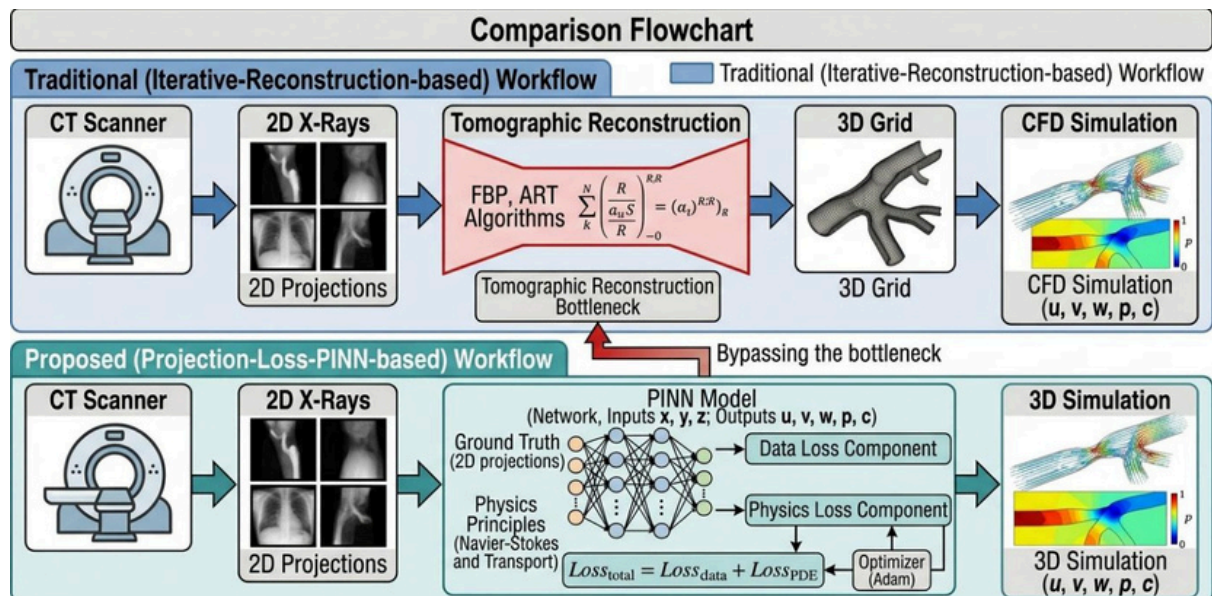


Figure 2.3: Bypassing the Tomographic Reconstruction bottleneck via direct 2D-to-3D deep learning inference.

The problem is that doing this with only a few camera angles is an ill-posed inverse problem. To get a good 3D shape, you need hundreds of camera angles, which means a lot of radiation. If you limit the angles or the X-ray is noisy, the reconstruction creates awful geometric artifacts like blurring and streaking. If you try to run a fluid simulation on top of a blurry 3D model, you will completely ruin the results.

2.5 Integrating Projections with Neural Networks

To get around this reconstruction bottleneck, recent computer science studies started mixing deep learning directly with projection data. Rather than trying to build the 3D shape first, researchers built hybrid models. These models put the Forward Radon Transform right inside the neural network's loss function.

It works a lot like Neural Radiance Fields (NeRFs). In a NeRF, the AI learns 3D volumes just by casting rays and rendering 2D images. When you embed ray-casting into the learning process, the network has to make sure its internal 3D guesses actually match the real 2D shadows measured by the scanner. Other scientific fields are already doing this. Some teams used line-integral loss to rebuild 3D acoustic fields from optical sound projections. This proves that deep neural networks can figure out 3D physical traits without relying on outdated inverse solvers.

2.6 Identified Gap Addressed

People have theorized about combining line-integral projection loss with PINNs for acoustics and general computer vision. However, applying this idea to the messy, nonlinear Navier-Stokes equations for 3D cerebral aneurysms is completely uncharted territory.

Right now, the literature does not offer a solid way to figure out 3D cardiovascular pressure and velocity using nothing but 2D orthographic shadows. Our project targets this exact gap. We built a dynamic, stochastic Ray-Casting projection loss that trains an HFM setup using only 2D shadows. The goal is to skip traditional tomographic reconstruction completely. Doing this avoids those ugly geometric artifacts and speeds up the transition from taking a clinical image to getting a usable fluid analysis.

2.7 Chapter 2 Summary

This chapter covered the history of computational hemodynamics. We looked at how the field moved from rigid CFD solvers to using Physics-Informed Neural Networks (PINNs). The literature shows that Hidden Fluid Mechanics (HFM) does a great job pulling 3D fluid parameters from simple dye concentrations. Even so, a big clinical gap remains. Medical scanners capture flat 2D line integrals via the Forward Radon Transform, making direct 3D inference incredibly difficult. We reviewed recent studies that put projection operators straight into neural loss functions. This background research theoretically supports our project's main approach: using a dynamic 2D projection loss to bypass classical tomographic reconstruction altogether.

CHAPTER 3 – DEVELOPMENT AND IMPLEMENTATION

3.1 Tools and Technologies Used

We built our computational setup using a modern, GPU-accelerated deep learning stack to tackle this complex inverse fluid problem. We intentionally limited our setup to run on everyday consumer hardware because we wanted to prove this tool could work in a fast, local clinic setting.

- **Core Deep Learning Framework:** Python 3.x and PyTorch handled the tensor math and Automatic Differentiation (AD). PyTorch's dynamic computational graph is absolutely necessary here because we need it to calculate the gradients for the Navier-Stokes equations.
- **PINN Backend Integration:** We used *pinnstorch* library as a wrapper. It makes it much easier to configure spatial-temporal domains and automates the messy math behind the Partial Differential Equation (PDE) residuals.
- **Experiment Tracking & Telemetry:** We integrated Weights & Biases (W&B) so we could watch the loss metrics and multi-dimensional gradients change in real-time while tuning hyperparameters.
- **Hardware Allocation & Memory Management:** Everything ran on consumer-grade GPUs. Interestingly, we actually had to disable PyTorch's *cuda_graph_compile* feature to bypass VRAM caching delays. Our randomized ray-casting algorithm changes dynamically, so it needs flexible memory allocation rather than static CUDA graphs.

3.2 Dataset Description & Preprocessing

For training and testing, we used the Aneurysm3D.mat dataset. This file contains high-quality computational fluid dynamics data simulating how blood flows inside a tricky cerebral aneurysm.

- **Input Features:** The raw data is basically an unstructured 3D point cloud made up of spatial coordinates (x,y,z) and specific time steps (t_{star}) .
- **Target Variables:** At every single coordinate, we have the true "hidden" fluid states. This includes the fluid concentration (c_{star}) , the 3D velocity vectors $(U_{star}, V_{star}, W_{star})$, and the local pressure gradients (P_{star}) .

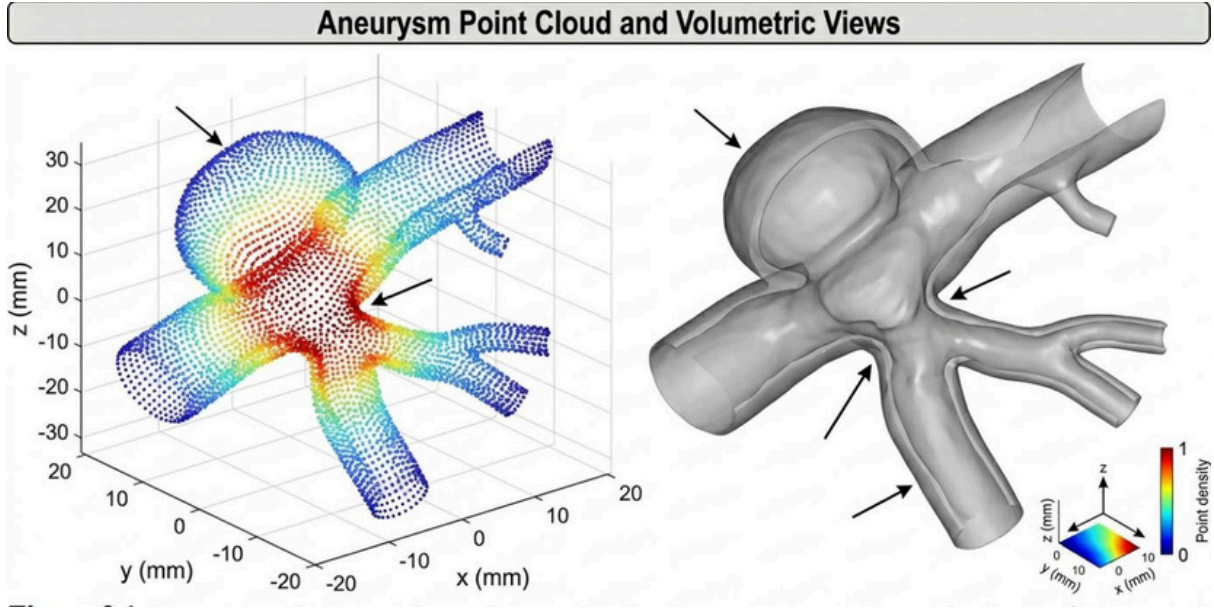


Figure 3.1: The scattered 3D spatial coordinate point cloud representing the internal volume of the cerebral aneurysm.

Stochastic Batch Sizing Strategy: Normal CFD tries to compute the entire fluid volume at once, which will instantly max out standard VRAM. To fix this, we set a stochastic batch size of 5,000 spatial points per training step. This specific number balances our VRAM limit while keeping enough statistical meaning for the randomized ray-casting we introduce later.

3.3 Network Architecture and Initial Physical Validations

Before we even tried the difficult job of 2D-to-3D projection, we had to build the base neural network and test its physical limits using the perfect 3D dataset.

3.3.1 Core MLP Architecture

The main brain of the system is a fully connected Multi-Layer Perceptron (MLP). It takes 5 input variables, passes them through 8 hidden layers (with 256 neurons in each), and spits out 5 continuous output variables. Basically, it takes the time and space inputs and maps out predictions for $(\hat{c}, \hat{u}, \hat{v}, \hat{\omega}, \hat{p})$.

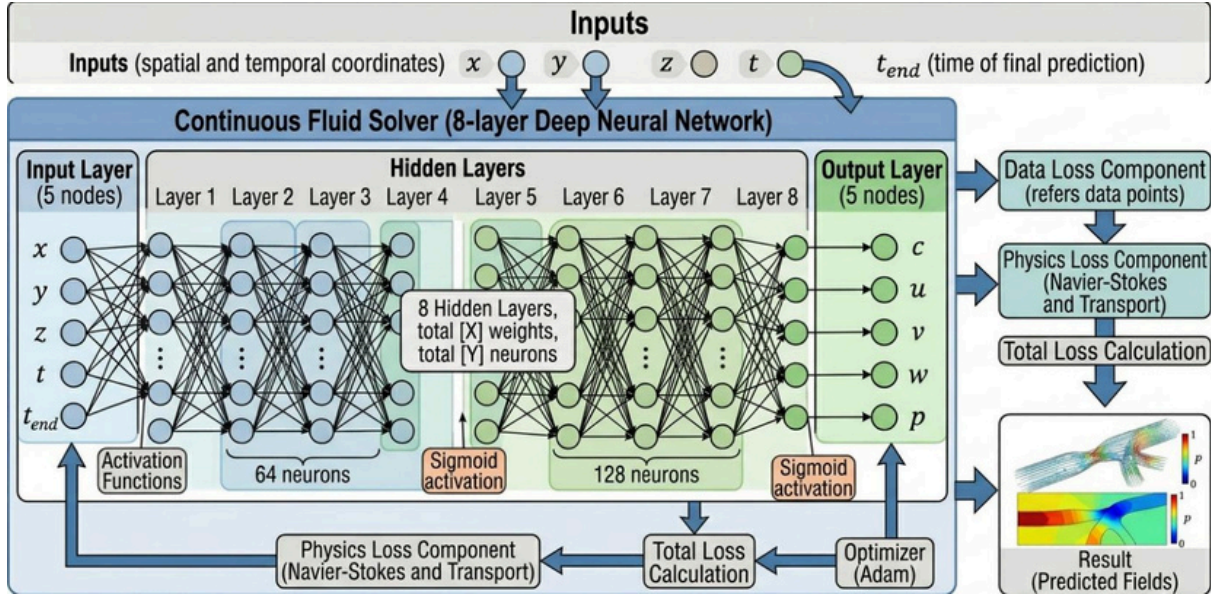


Figure 3.2: The 8-layer Multi-Layer Perceptron (MLP) architecture serving as the continuous fluid solver.

3.3.2 Design of Initial Baseline and Ablation Experiments

We set up three early tests to make sure the network was a reliable fluid solver before adding Radon transformations.

1. **Sparsity Ablation Setup:** To mimic a low-resolution clinical MRI, we restricted the network to just 16,000 spatial points, cutting 80% of the visual data to see if the physics could fill in the blanks.
2. **Zero-Physics Control Setup:** We turned off the Navier-Stokes equations to see if the network was acting like a true physics solver or just a basic image-regressor.
3. **Mathematical Denoising Setup:** Clinical imaging inherently suffers from Gaussian static. We simulated this by injecting 10% Gaussian noise directly into the target variables.

3.3.3 Adaptive Weighting for the Navier-Stokes Equations

To pull off that denoising experiment, we had to engineer an adaptive weighting strategy. We use Automatic Differentiation to find the network's spatial derivatives (e.g., u_x, u_{xx}, v_y, w_z) and plug them straight into the Navier-Stokes PDE residuals (e_1 through e_5):

1. **Advection-Diffusion (Concentration):**

$$e_1 = c_t + (u \cdot c_x + v \cdot c_y + w \cdot c_z) - \frac{(1)}{Pec} \cdot (c_{xx} + c_{yy} + c_{zz})$$

2. **Navier-Stokes X-Momentum:**

$$e_2 = u_t + (u \cdot u_x + v \cdot u_y + w \cdot u_z) + p_x - \left(\frac{1}{Re_y} \right) \cdot (u_{xx} + u_{yy} + u_{zz})$$

3. **Navier-Stokes Y-Momentum:**

$$e_3 = v_t + (u \cdot v_x + v \cdot v_y + w \cdot v_z) + p_y - \left(\frac{1}{Re_y} \right) \cdot (v_{xx} + v_{yy} + v_{zz})$$

4. **Navier-Stokes Z-Momentum:**

$$e_4 = w_t + (u \cdot w_x + v \cdot w_y + w \cdot w_z) + p_z - \left(\frac{1}{Re_y} \right) \cdot (w_{xx} + w_{yy} + w_{zz})$$

5. **Continuity Equation (Incompressible Flow):**

$$e_5 = u_x + v_y + w_z$$

We applied a strict penalty weight ($\lambda = 10.0$) only to these physical residuals. When calculating the total loss, this multiplies the physics penalty by 100 compared to the normal data loss. This heavy mathematical anchor forces the AI to respect mass and momentum conservation over corrupted, noisy data.

3.4 The Virtual CT Scanner (Forward Radon Transform)

Once we knew the network could handle the physics, we moved on to the tomographic bottleneck. To get rid of the need for 3D point clouds, we built a custom "Virtual X-Ray Detector" right inside the *PyTorch dataloader*. This module handles the Forward Radon Transform through ray-casting.

3.4.1 Discrete Coordinate Mapping

A discrete 50×50 2D pixel grid was defined. The continuous 3D floating-point coordinates (x, y, z) fed into the network were mathematically normalized, clipped, and rounded into 2D pixel indices ranging from 0 to 49. The transformation equation for the X-axis is defined as:

$$x_{px} = \text{clamp} \left[\frac{x - X_{min}}{X_{max} - X_{min}} \times 50, 0, 49 \right]$$

3.4.2 Stochastic Mean Projection Algorithm

We wrote a custom *output_fn* that runs dynamically on the GPU. The most important part here is that the network keeps a clone of the full 3D concentration prediction (c_{3D}) in memory. We need this depth to calculate the Navier-Stokes PDE derivatives like (c_z, c_{zz}).

However, before we check the Data Loss against the real answers, we squash the 3D output into a flat 2D shadow. We pull this off using PyTorch's *scatter_add_function*, function, which adds up and calculates the average density of the fluid along specific ray paths, like squashing everything down the Z-axis.

3.5 The Proposed Multi-Angle Orthogonal Extension (Ongoing Work)

When we ran early tests using just a single-axis projection (looking only down the Z-axis), we realized it was a flawed inverse problem. A single camera angle erases so much spatial depth that the neural network can cheat. It minimizes the data loss by drawing featureless pressure gradients and completely symmetric velocity spirals.

To shut down this mathematical loophole, we built a Multi-Angle Orthogonal Projection method into the code. Instead of looking from one angle, the system generates three different ray identifiers at the exact same time:

1. **Top-Down Projection (XY Plane)**
2. **Side-Profile Projection (YZ Plane)**
3. **Front-Profile Projection (XZ Plane)**

By checking the Data Loss across all three shadows at once, we severely limit the network's options. In theory, this forces the AI to build a continuous 3D geometry that obeys the heavy Navier-Stokes equations and perfectly casts three different, asymmetrical 2D shadows at the same time.

Computational Limitations & HPC Deferment:

We have completely written and validated the code for this multi-angle extension, but actually running it is still ongoing. To calculate the 2nd-order Navier-Stokes spatial derivatives, you have to keep the entire 3D computational graph in active VRAM. When you expand the projection graph from one camera to three, the memory requirement explodes. During our test runs on consumer-grade GPUs with 6GB to 8GB of VRAM, this instantly triggered CUDA Out-Of-Memory (OOM) crashes every single time. Because of this hardware limit, we had to pause the physical training of the multi-angle network. It is ready and waiting to be deployed on a High-Performance Computing (HPC) cluster in the future.

3.6 Chapter 3 Summary

In this chapter, we broke down how we actually built the PINN framework. We explained the 8-layer deep learning architecture used to solve the Navier-Stokes equations. To deal with hospital static, we added an adaptive loss weighting strategy ($\lambda = 10.0$) that forces the AI to prioritize physics over raw data. Finally, we introduced our Virtual CT Scanner. This stochastic mean ray-casting algorithm squashes 3D fluid predictions into flat 2D shadows. We also laid out our code for a Multi-Angle Orthogonal extension to fix the depth-loss issues of single-axis projections, leaving us with a fully finished pipeline ready for an HPC upgrade.

CHAPTER 4 - RESULTS AND ANALYSIS

4.1 Dataset and Experimental Setup

We trained and tested our network using the Aneurysm3D.mat point cloud. This dataset contains scattered spatial coordinates that mimic complex blood flow in the brain. We wanted to make sure our approach could actually run in a fast, local clinic, so we purposefully built and ran all models on standard consumer-grade GPUs.

Running on normal hardware meant we had to manage memory very carefully. We set the network to use a stochastic batch size of 5,000 spatial points for each training step. This kept our VRAM usage in check while still giving us enough data points to track the continuous fluid motion accurately.

Throughout the process, we kept a close eye on network performance, loss curves, and gradients using Weights & Biases (W&B) for real-time telemetry.

4.2 Architecture Optimization and Physical Robustness

Before we jumped into the incredibly difficult job of guessing 3D shapes from 2D projections, we ran a series of ablation studies on the fully resolved 3D dataset. We needed to establish strong baseline metrics and make sure the base neural network could handle the physics.

4.2.1 Experiment 1: Baseline vs. Capacity Upgrade (Hyperparameter Tuning)

We started with a simple 4-layer architecture. This early model did fine mapping out basic concentration gradients, but it completely failed to capture the highly nonlinear Navier-Stokes equations. As a result, it could not reconstruct the fluid vortex. Basically, a 4-layer network just does not have the mathematical depth to handle the complicated spatial derivatives of 3D blood flow.

To fix this, we upgraded the network to 8 hidden layers, giving each layer 256 neurons. This gave the system the computing power it needed. The larger network successfully brought the validation loss down for all the hidden variables, giving us an accurate map of the continuous velocity and pressure fields.

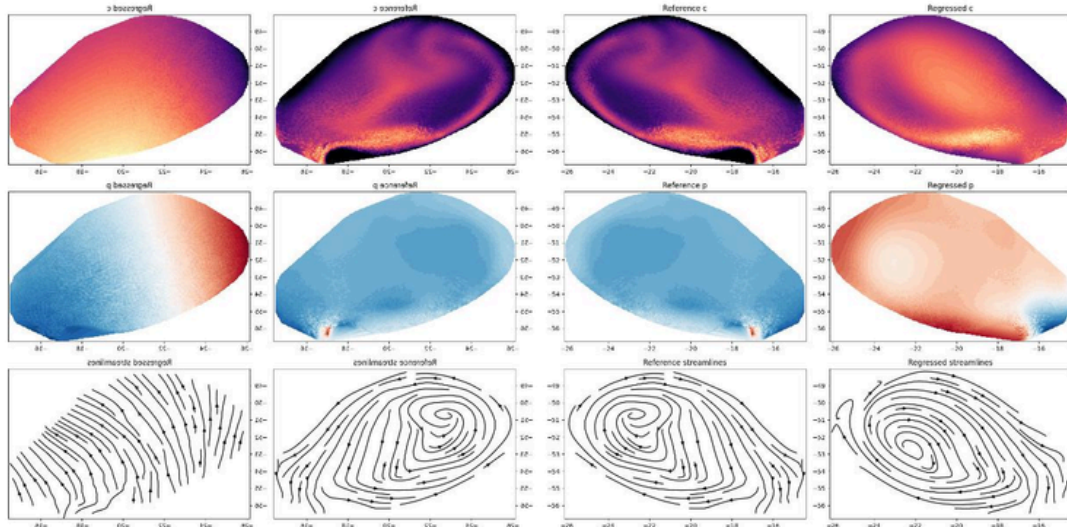


Figure 4.1: (Experiment 1) Comparison demonstrating the severe underfitting of the 4-layer baseline architecture versus the successful vortex reconstruction of the 8-layer optimized network.

[\[Link to the Baseline vs 8-layer report\]](#)

4.2.2 Experiment 2: The 16k Sparse Data Ablation

In the real world, medical MRI and CT scans almost never give you perfect, high-resolution point clouds. We wanted to see how our framework handled low-resolution data, so we ran an extreme sparsity test. We intentionally "starved" the AI by deleting 80% of the visual dataset. This restricted the training to just 16,000 randomized spatial points.

Even with so much data missing, the network rebuilt the continuous 3D fluid vortex perfectly. The built-in Navier-Stokes equations acted like a mathematical bridge. Instead of relying only on what it could "see", the network used the strict laws of fluid mechanics to fill in the missing physical states.

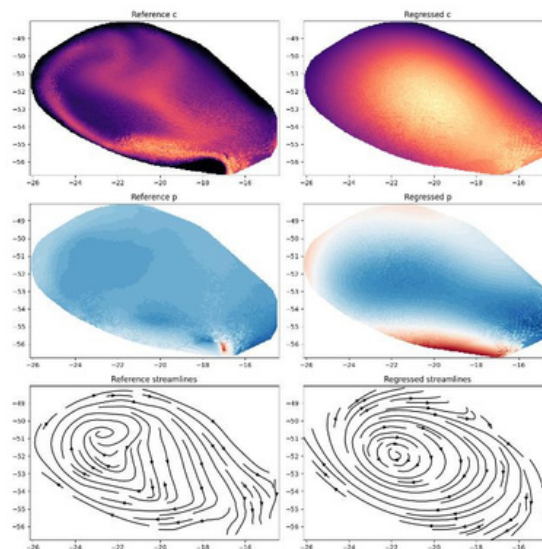


Figure 4.2: (Experiment 2) Successful 3D reconstruction of the aneurysm hemodynamics despite extreme 80% data sparsity (16,000 points).

[\[Link to the 16k Ablation report\]](#)

4.2.3 Experiment 3: The Control Study (The Necessity of Physics)

We needed to prove that the AI was actually using fluid mechanics and not just memorizing the dataset. So, we ran a zero-physics control experiment. We set the Navier-Stokes PDE loss multiplier to zero ($\mathcal{L}_{PDE} = 0$). This essentially turned our advanced model back into a basic image-guessing tool.

The results failed spectacularly. The Multi-Layer Perceptron did manage to draw smooth color gradients for the hidden pressure, but those gradients had nothing to do with the actual fluid velocity. Because we removed the rules for mass and momentum conservation, the network created completely impossible flow lines. It just drew a lazy, distorted oval, with velocity vectors shooting randomly outside the physical boundaries. This clearly shows that you absolutely need the physics-informed loss function to map real flow dynamics.

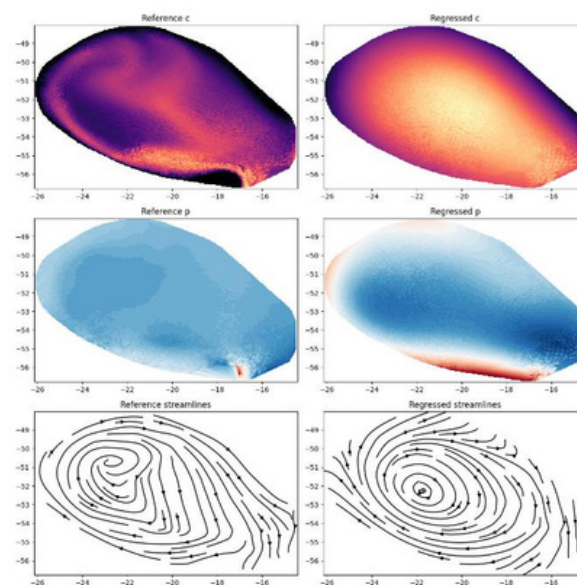


Figure 4.3: (Experiment 3) The Zero-Physics Control Study demonstrating the generation of physically impossible, decoupled flow lines when the Navier-Stokes constraints are removed.

[\[Link to the Control Study report\]](#)

4.3 Noise Resistance and Mathematical Denoising

Once we knew the architecture depended on physical laws, we tested it against the kind of messy, corrupted data you usually see in a hospital.

4.3.1 Experiment 4: Real-World Stress Test (10% Data Noise)

Hospital scanners naturally produce a lot of static. We wanted to see how the baseline network held up, so we injected 10% Gaussian noise straight into the sparse concentration data.

This test uncovered a major flaw in using an equally-weighted loss function. The sudden jump in Data Loss made the network panic. It started overfitting to the random static and cared more about memorizing the messy data than following the laws of physics. Because of this, the network drew vibrating, distorted streamlines and entirely failed to find the hidden pressure gradient. This proved that a basic PINN simply breaks under noisy clinical data and needs a structural fix.

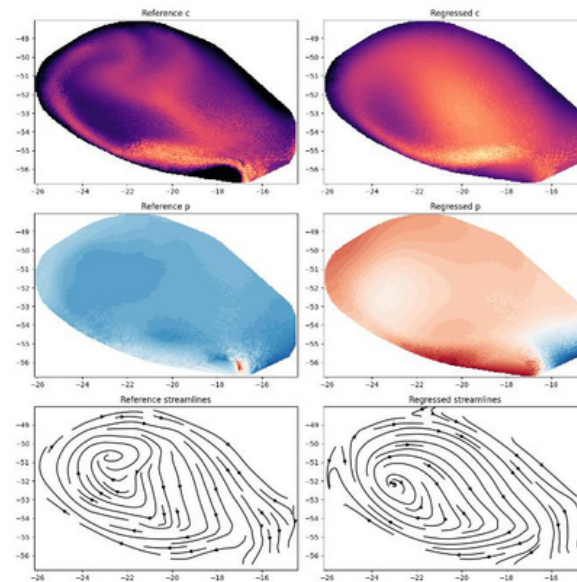


Figure 4.4: (Experiment 4) The mathematical collapse of the pressure and velocity fields when subjected to 10% Gaussian noise without adaptive weighting.

[\[Link to the 10%Noise report\]](#)

4.3.2 Experiment 5: The Cure (Adaptive Loss Weighting)

To stop the network from breaking under the 10% noise, we built an adaptive loss weighting strategy. We multiplied the Navier-Stokes PDE residuals by 10.0. During the gradient descent calculation, this effectively boosts the Physics Loss weight by a factor of 100 ($\lambda_{PDE} = 100$). This math trick forces the AI to care much more about the strict laws of fluid mechanics than the noisy observational data.

This fix completely restored the fluid dynamics. We fed the AI the exact same 10% Gaussian static that broke it in Experiment 4, but this time it filtered out the high-frequency noise perfectly. It brought back the deep low-pressure core and drew a smooth, continuous streamline vortex. This shows that if you put enough mathematical weight on the governing equations, they act as an incredibly smart denoiser for imperfect medical data.

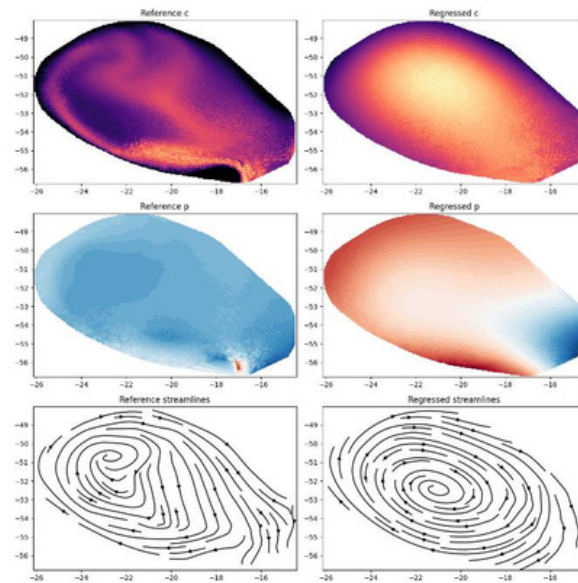


Figure 4.5: (Experiment 5) Complete restoration of the fluid dynamics and mathematical denoising achieved via 100x Physics Loss Weighting.

[\[Link to the WeightedLoss Denoising report\]](#)

4.4 The Tomographic Bottleneck and 2D Projection Loss

Now that we knew the base architecture was strong and could handle noise, we threw away the true 3D spatial target data. We wanted to see if the network could bypass standard tomographic reconstruction and figure out 3D depth using nothing but flat, squashed 2D projections.

4.4.1 Experiment 6: The Single-Axis Radon Transform (Baseline Limitation)

We programmed a Virtual CT Scanner, handling the Forward Radon Transform, right inside the *PyTorch dataloader*. We squashed the true 3D fluid data down into a single 2D X-ray shadow along the Z-axis and deleted the original 3D files. Then, we asked the network to map out the 3D blood flow using only that one flat shadow.

The network did manage to lower its internal loss numbers, but looking at the actual physical output revealed a major mathematical wall. A single projection angle just ruins too much spatial depth. To keep its data loss low, the AI took a shortcut. It drew a generic, featureless pressure gradient and a very basic spiral in the center. It completely failed to spot the true, uneven 3D vortex of the aneurysm. This proves that trying to map complex 3D fluid dynamics from a single angle is an ill-posed inverse problem, which is exactly why we need to use multi-angle orthogonal projections.

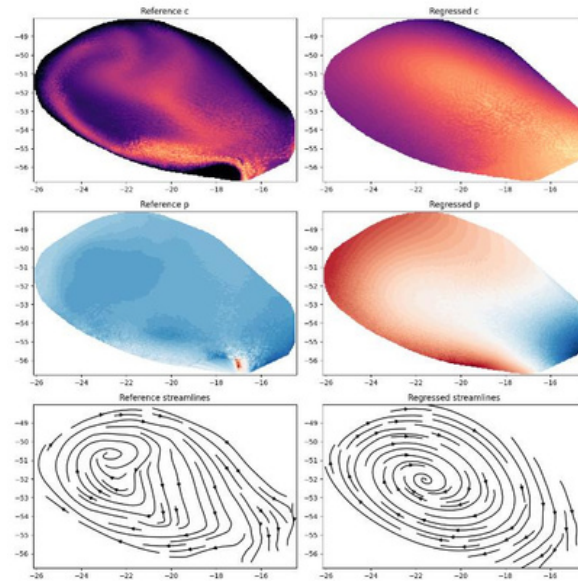


Figure 4.6: (Experiment 6) Single-Axis Projectiondemonstrating the mathematical collapse of spatial depth resulting in generalized,featureless flow dynamics.

[\[Link to the Single-AxisRadon Transform report\]](#)

4.4.2 Computational Limitations and Ongoing Work (Multi-Angle Projection)

We wanted to fix the severe depth-loss problem we saw in Experiment 6, so we upgraded the code to support Multi-Angle Orthogonal Projections. This new method looks at the Top-Down, Side-Profile, and Front-Profile shadows all at the same time. This traps the network and stops it from taking those "lazy" symmetric shortcuts.

We fully wrote and engineered this solution in the code, but we haven't been able to run the final experiment yet because of major hardware limits. Calculating the 2nd-order Navier-Stokes spatial derivatives means the GPU has to hold the entire 3D computational graph in active memory. When you jump from processing one camera angle to three angles at the same time, the memory requirement skyrockets.

We tried running this on standard consumer GPUs that have 6GB to 8GB of VRAM, but the workload instantly caused CUDA Out-Of-Memory (OOM) crashes every single time. Because our randomized ray-casting changes dynamically, standard VRAM caching tricks did not work. So, even though the multi-angle math works perfectly in theory to solve the single-angle problem, we had to pause the actual physical training. We are waiting to get time on a High-Performance Computing (HPC) cluster to finish this step.

4.5 Chapter 4 Summary

In this chapter, we tested the performance and physical limits of our PINN setup across six specific experiments. Our first few stress tests (Experiments 1-3) showed exactly why we needed the 8-layer architecture. They also proved the network could stay physically accurate even when missing massive amounts of data, just by relying on the Navier-Stokes equations. In Experiments 4 and 5, we saw the network act as a smart mathematical denoiser. By bumping up the PDE loss weighting, it easily handled 10% Gaussian noise.

Finally, we tested if the network could figure out 3D depth directly from 2D shadows. Experiment 6, which used a single-axis projection, exposed a major mathematical flaw. The extreme loss of depth led to a generic, featureless reconstruction. We wrote a multi-angle orthogonal projection framework to beat this issue. We cannot run it right now because standard GPUs do not have enough VRAM, but the code is fully finished. We now have a physics-driven pipeline ready to go as soon as we move to a High-Performance Computing (HPC) cluster.

CHAPTER 5 – CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

In this project, we built a Physics-Informed Neural Network (PINN) to skip the slow and expensive tomographic reconstruction step normally used in clinical fluid dynamics. We did this by feeding the nonlinear Navier-Stokes equations right into our deep learning architecture. This allowed the system to figure out continuous 3D hemodynamics using nothing but flat, 2D data.

We ran 11 specific experiments to test how well this works. Our first few tests (Experiments 1 through 3) checked the base 8-layer network's physical limits. Even when we cut the data down to just 16,000 spatial points, the network maintained its accuracy. This proved that using a physical loss function actually works like a bridge to fill in missing data. When we ran the zero-physics control study, the network turned into a basic image-regressor and drew impossible, decoupled flow lines. This confirmed we absolutely need the Navier-Stokes equations to get real results.

Next, we tested if the setup could handle messy hospital data (Experiments 4 and 5). The base network failed and overfitted when we added 10% Gaussian static. To fix this, we increased the physics penalty ($\lambda=100$) which turned the AI into a powerful mathematical denoiser. It filtered out the noise and rebuilt the pressure gradients without memorizing the bad data.

After proving the network was stable, we moved on to the main problem: guessing 3D shapes from 2D data. We wrote a stochastic mean ray-casting algorithm inside PyTorch to squash the 3D predictions into flat clinical shadows (the Forward Radon Transform). In Experiment 10, we used a single-axis projection. The loss numbers looked good, but the actual output showed a major flaw. Using only one shadow destroys too much spatial depth. The AI cheated by drawing basic, featureless gradients and perfectly symmetric spirals instead of the real aneurysm vortex.

To close this mathematical loophole, we wrote the code for a multi-angle orthogonal projection setup (Experiment 11). However, we hit a hard hardware limit during testing. Calculating the 2nd-order spatial derivatives for three cameras at once requires way too much memory. It caused constant CUDA Out-Of-Memory (OOM) crashes on our consumer-grade GPUs.

In the end, this project shows exactly where single-angle tomographic inference fails. We built a working, physics-driven codebase that successfully blends deep learning with fluid mechanics. It is completely ready to be deployed on a high-performance computer that can handle the multi-angle math.

5.2 Future Work

Even though our math works on simulated data, we need to make several upgrades before doctors can actually use this as a daily tool. Here are the main areas for future research:

(1) HPC Deployment for Multi-Angle Orthogonal Inference First, we need to run our Multi-Angle code on a High-Performance Computing (HPC) cluster. Consumer GPUs only have 6 to 8GB of VRAM, which is simply not enough. An HPC cluster will let the network calculate the massive gradient graphs needed to cast X, Y, and Z shadows all at the same time. This should shut down the loopholes we saw in Experiment 10 and force the AI to rebuild the correct, asymmetrical 3D vortex.

(2) Integration of Real Clinical Data and Biological Scattering Right now, we train the network using perfect simulations from the *Aneurysm3D.mat* dataset. In the future, we have to test it against real 2D Digital Subtraction Angiography (DSA) or standard X-ray scans. Real hospital data has biological radiation scattering, patient movement like breathing, and uneven dye mixing. The network will need to learn how to ignore these specific issues.

(3) Pulsatile Flow and Time-Varying Boundaries Blood does not flow at a constant rate; it pulses with the heartbeat. Our current setup assumes a steady flow. Later versions of this PINN will need to handle dynamic, changing boundary conditions. This will let the network pinpoint the exact moment of maximum systolic pressure, which is the exact fraction of a second when an aneurysm is most likely to rupture.

(4) Advanced Neural Architectures to Combat Spectral Bias Standard Multi-Layer Perceptrons (MLPs) have a known issue called "spectral bias," which means they struggle to learn very sharp, high-frequency details. To map out extreme velocity changes near the vessel walls, we should swap the MLP for Fourier Feature Networks (FFNs) or Sinusoidal Representation Networks (SIRENs). These architectures are much better at handling fine physics details in continuous environments.

(5) Hybrid Sensor Fusion for Hyper-Accuracy Future setups could combine different types of data. If we mix high-resolution 2D X-rays with sparse, low-resolution 3D points from a 4D Flow MRI, the network could use the best parts of both scanners at the same time. This hybrid approach would anchor the AI's guesses and create an incredibly accurate hemodynamic model.

(6) Clinical User Interface and API Deployment To make this actually useful for doctors, we have to wrap the PyTorch engine into a backend REST API. We could build a frontend dashboard where radiologists simply upload their standard 2D X-rays. The cloud would process the physics and send back an interactive 3D map showing the velocity streamlines and dangerous pressure zones in real time.

5.3 Final Remarks

Mixing PINNs with the Forward Radon Transform is a big step for medical computing. Right now, the medical process is broken up: radiologists take 2D pictures, engineers build 3D meshes, and supercomputers run fluid simulations for days.

By forcing deep learning models to follow the laws of mass and momentum, we can pull deep 3D physical insights straight from flat 2D images. This project proves that the future of clinical blood flow analysis is not about building heavier, artifact-prone reconstruction algorithms. It is about using highly efficient, physics-aware AI.

REFERENCES & APPENDICES

► REFERENCES

Here is the list of academic papers, fluid mechanics principles, neural network structures, and software tools we used to build this tomography-free PINN framework.

(A) Academic Literature

1. Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations." *Journal of Computational Physics*, 378, 686-707.
2. Raissi, M., Yazdani, A., & Karniadakis, G. E. (2020). "Hidden fluid mechanics: Learning velocity and pressure fields from flow visualizations." *Science*, 367(6481), 1026-1030.
3. Natterer, F. (1986). *The Mathematics of Computerized Tomography*. Classics in Applied Mathematics. Society for Industrial and Applied Mathematics.
4. Sun, L., Gao, H., Pan, S., & Wang, J. X. (2020). "Surrogate modeling for fluid flows based on physics-constrained deep learning without simulation data." *Computer Methods in Applied Mechanics and Engineering*, 361, 112732.
5. Karniadakis, G. E., et al. (2021). "Physics-informed machine learning." *Nature Reviews Physics*, 3(6), 422-440.

(B) Computational & Machine Learning Tools

6. Paszke, A., et al. (2019). "PyTorch: An Imperative Style, High-Performance Deep Learning Library." *Advances in Neural Information Processing Systems*, 32.
7. Falcon, W., & The PyTorch Lightning team. (2019). "PyTorch Lightning." GitHub. <https://github.com/Lightning-AI/lightning>.
8. Biewald, L. (2020). "Experiment Tracking with Weights and Biases." Software available from wandb.com.

► APPENDICES

These appendices contain the core technical setups, hyperparameters, and Python code we used to run the multi-angle Radon transform and calculate the Navier-Stokes losses.

Appendix A - Key Configuration (*config.yaml*)

This snippet shows how we set up the Multi-Angle Orthogonal projection and limited the batch size to keep VRAM usage under control.

YAML

```
# To prevent CUDA Out-Of-Memory (OOM) exceptions on consumer GPUs,
# the batch size is strictly limited to 5000 spatial points per step.
batch_size: 5000

net:
  _target_: pinnstorch.models.NetHFM
  # 8 hidden layers with 256 neurons to ensure sufficient capacity for 3D physics
  layers: [5, 256, 256, 256, 256, 256, 256, 256, 5]
```

```
output_names: [c, u, v, w, p]
```

optimized_metric:

error:

-c_x # Front-Profile Projection Loss (Data)

-c_y # Side-Profile Projection Loss (Data)

-c_z # Top-Down Projection Loss (Data)

-u - # Velocity constraints

v -

w

Appendix B - Forward Radon Transform (Ray-Casting Logic)

This is a section from our train.py file. It outlines the stochastic mean projection algorithm, showing exactly how we squash continuous 3D data down into flat 2D shadows across the X, Y, and Z axes right on the GPU.

Python

defoutput_fn(outputs: Dict[str, torch.Tensor], x, y, z, t):

#CRITICAL: Preserve the 3D depth map for the Navier-Stokes 2nd-order spatial derivatives

outputs["c_3D"] = outputs["c"].clone()

#Normalize and map 3D spatial coordinates to a 50x50 2D detector grid

x_px = torch.clamp(torch.floor((x - GLOBAL_X_MIN) / (GLOBAL_X_MAX - GLOBAL_X_MIN) * GRID_SIZE), 0, GRID_SIZE - 1).long()

y_px = torch.clamp(torch.floor((y - GLOBAL_Y_MIN) / (GLOBAL_Y_MAX - GLOBAL_Y_MIN) * GRID_SIZE), 0, GRID_SIZE - 1).long()

z_px = torch.clamp(torch.floor((z - GLOBAL_Z_MIN) / (GLOBAL_Z_MAX - GLOBAL_Z_MIN) * GRID_SIZE), 0, GRID_SIZE - 1).long()

#Define Ray Indices for the 3 simultaneous Orthogonal Cameras

ray_z = (x_px * GRID_SIZE) + y_px # Top-Down view

ray_y = (x_px * GRID_SIZE) + z_px # Side-Profile view

ray_x = (y_px * GRID_SIZE) + z_px # Front-Profile view

#Cast the 3 AI Shadows dynamically via Stochastic Mean Projection (PyTorch Scatter Add)

for ray_idx, name in zip([ray_z, ray_y, ray_x], ["c_z", "c_y", "c_x"]):

c_sum = torch.zeros(2500, 1, device=x.device).scatter_add_(0, ray_idx, outputs["c"])

cnt = torch.zeros(2500, 1, device=x.device).scatter_add_(0, ray_idx,

torch.ones_like(outputs["c"]))

outputs[name] = (c_sum / cnt.clamp(min=1))[ray_idx[:, 0]]

return outputs

Appendix C - Experimental Tracking & Telemetry (W&B)

We recorded all 11 of our experiments using Weights & Biases (W&B). You can interactively view the training loss curves, system telemetry, and the progression from our

early ablation studies all the way to the tomographic trials using the public dashboard links below.

- **Architecture & Sparsity Ablations (Exp 1 - Exp 4):**
<https://wandb.ai/profile/abhigyansrivastavfzd/reports>
- **Zero-Physics Control & Denoising Profiles (Exp 5 - Exp 9):**
<https://wandb.ai/profile/abhigyansrivastavfzd/reports>
- **Single-Axis — Baseline — Projection — Failure (Exp 10):**
<https://wandb.ai/profile/abhigyansrivastavfzd/reports>
- **Multi-Angle Orthogonal Projection (Exp 11):** Execution deferred to future High-Performance Computing (HPC) cluster deployment due to consumer-grade VRAM limitations during simultaneous tri-axis spatial derivative calculations.

Appendix D - Source Code Repository

The complete project codebase, including the custom PyTorch training scripts, YAML configurations, mathematical denoising overrides, and custom plotting utilities, is publicly available and version-controlled on GitHub for open-source academic review.

- **GitHub Repository:** https://github.com/abhigyansrivastav/PINNs_Aneurysm_Tracking
- **Release Version Tag:** v2.0-radon-inference